Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы трансляции»

ОТЧЕТ
к лабораторной работе №4
на тему:
«СЕМАНТИЧЕСКИЙ АНАЛИЗАТОР»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Описание работы программы 4
3 Ход выполнения программы 5
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

На основе результатов лексического и синтаксического анализа разработать семантический анализатор для языка Idris. Анализатор должен проверять корректность использования идентификаторов и типов, формировать таблицу символов и выдавать сообщения о семантических ошибках. Входные данные считываются из файла input.txt.

2 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ

В лабораторной работе был реализован семантический анализатор для подмножества языка Idris. В качестве базовой части были использованы ранее разработанные лексический и синтаксический анализаторы. Семантический анализатор получает на вход поток токенов и набор логических строк, построенных синтаксическим анализатором, после чего формирует таблицу символов и выполняет контроль типов.

В программе выделены структуры FunctionType и SymbolEntry, а также класс SemanticAnalyzer. Он хранит информацию о глобальных функциях, локальных переменных, параметрах функций и специальных встроенных операциях. В таблицу символов заносятся имена, область видимости, определённые типы и номера строк исходного файла.

Семантический анализ включает нормализацию записей типов, разбор сигнатур функций, проверку соответствия фактических и ожидаемых типов аргументов, анализ let-связываний, условий if, ветвей case и операторов внутри do-блоков. Для встроенных функций putStrLn, show, cast, insert и значения empty задаются специальные правила интерпретации типов.

Отдельное внимание уделено совместимости типов. Анализатор допускает безопасные числовые преобразования, различает списки, кортежи, символьные и строковые литералы, а также проверяет корректность шаблонов сопоставления. При обнаружении несоответствий формируются сообщения с указанием строки и позиции в исходном тексте.

В результате работы программы выводятся таблица токенов, дерево разбора, таблица семантических символов, а также три списка ошибок: лексические, синтаксические и семантические. Такой формат позволяет поэтапно отследить состояние программы и увидеть, на каком именно уровне анализа возникает ошибка.

3 ХОД ВЫПОЛНЕНИЯ ПРОГРАММЫ

Проверка семантического анализатора проводилась на файле input.txt. В тестовой программе содержатся объявления и вызовы функций, условные выражения, типизированные let-объявления, структуры Vect и SortedMap, а также заведомо некорректные строки с лексическими ошибками. Анализатор успешно сформировал таблицу символов и не обнаружил семантических ошибок в корректной части программы.

Сформированная таблица символов содержит глобальные сигнатуры numbersList, checkValue, processData, forLoop и main, а также локальные переменные и параметры x, d, n, id, name, vec, k, result, myPair, myVec, myChar, bigInt, bVal, dict. Для каждого идентификатора указаны область видимости, тип и номер строки.

При выполнении тестов программа вывела сообщения No syntax errors found и No semantic errors found, что подтверждает корректность семантического разбора для заданного входа. Лексические ошибки в строках error1–error4 были зафиксированы отдельно и не помешали анализу оставшейся части файла.

Результаты показывают, что анализатор корректно проверяет типы выражений, параметры функций, ветви case и операторы do-блоков. Полученная семантическая информация может быть использована далее для интерпретации программы и контроля корректности выполнения. Вывод продемонстрирован ниже, на рисунке 3.1.

```
================================================================================================
SEMANTIC SYMBOL TABLE
================================================================================================
SCOPE            NAME             TYPE                                        LINE
------------------------------------------------------------------------------------------------
global           numbersList      List Int                                    4
global           checkValue       Int -> Double -> String                     7
global           processData      (Int, String) -> Vect 3 Int -> String       14
global           forLoop          Nat -> IO ()                                20
global           main             IO ()                                       26
checkValue       x                Int                                         8
checkValue       d                Double                                      8
let              n                Nat                                         9
processData      id               Int                                         15
processData      name             String                                      15
processData      vec              Vect 3 Int                                  15
forLoop          k                Nat                                         22
main             result           String                                      28
main             myPair           (Int, String)                               31
main             myVec            Vect 3 Int                                  32
main             myChar           Char                                        33
main             bigInt           Integer                                     36
main             bVal             Bits8                                       37
main             dict             SortedMap String Int                        38
```

Рисунок 3.1 – Вывод семантической таблицы

ВЫВОД

В ходе выполнения лабораторной работы разработан семантический анализатор для подмножества языка Idris, интегрированный с ранее созданными лексическим и синтаксическим анализаторами. Анализатор выполняет проверку корректности использования идентификаторов и типов на основе синтаксического дерева и логических строк, формирует таблицу символов с указанием области видимости, типа и строки объявления для каждого имени. Реализована система правил совместимости типов, включающая числовые продвижения (Int, Nat, Integer, Bits8 к Double), проверку параметров функций, условных выражений, let-связываний, case-ветвей и do-блоков. Встроенные функции (putStrLn, show, cast, insert) и предопределённые значения (empty) зарегистрированы с соответствующими сигнатурами. Анализатор различает списки, кортежи, параметризованные типы (Vect, SortedMap) и проверяет согласованность их элементов. При обнаружении семантических ошибок формируются понятные сообщения с указанием строки и позиции, что облегчает отладку. Тестирование на файле input.txt подтвердило отсутствие семантических ошибок в корректной части программы и успешное построение полной таблицы символов для всех глобальных и локальных идентификаторов. Полученное решение создаёт основу для следующего этапа трансляции – интерпретации исходного кода или генерации целевого кода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Грис, Д. Конструирование компиляторов для цифровых вычислительных машин: Мир, 1975. – 544 с.

[2] Серебряков, В. А. Основы конструирования компиляторов: Издательский центр «Академия», 2009. – 352 с.

[3] Документация по Idris [Электронный ресурс]. – Режим доступа: https://docs.idris-lang.org/en/latest. – Дата доступа: 11.02.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация семантического анализатора

```
class SemanticAnalyzer
{
private:
    const vector<Token> &tokens;
    const vector<LogicalLine> &lines;
    map<string, FunctionType> functionTable;
    map<string, string> globalValues;
    vector<SymbolEntry> symbols;
    vector<string> semanticErrors;

    static bool hasLexicalError(const LogicalLine &line)
    {
        for (const auto &token : line.tokens)
        {
            if (token.kind == TokenKind::Error)
            {
                return true;
            }
        }
        return false;
    }

    static bool isFullyWrapped(const vector<Token> &slice, const string
&openToken, const string &closeToken)
    {
        if (slice.size() < 2U || slice.front().value != openToken ||
slice.back().value != closeToken)
        {
            return false;
        }

        int depth = 0;
        for (size_t i = 0; i < slice.size(); ++i)
        {
            if (slice[i].value == openToken)
            {
                depth++;
            }
            else if (slice[i].value == closeToken)
            {
                depth--;
                if (depth == 0 && i + 1 != slice.size())
                {
                    return false;
                }
            }
        }

        return depth == 0;
    }

    static int findTopLevelToken(const vector<Token> &slice, const set<string>
&targets, bool searchFromRight = false)
    {
        int parenDepth = 0;
```

```
        int bracketDepth = 0;

        if (!searchFromRight)
        {
            for (size_t i = 0; i < slice.size(); ++i)
            {
                const string &value = slice[i].value;
                if (value == "(")
                {
                    parenDepth++;
                    continue;
                }
                if (value == ")")
                {
                    parenDepth--;
                    continue;
                }
                if (value == "[")
                {
                    bracketDepth++;
                    continue;
                }
                if (value == "]")
                {
                    bracketDepth--;
                    continue;
                }
                if (parenDepth == 0 && bracketDepth == 0 && targets.count(value)
!= 0U)
                {
                    return static_cast<int>(i);
                }
            }
            return -1;
        }

        for (int i = static_cast<int>(slice.size()) - 1; i >= 0; --i)
        {
            const string &value = slice[static_cast<size_t>(i)].value;
            if (value == ")")
            {
                parenDepth++;
                continue;
            }
            if (value == "(")
            {
                parenDepth--;
                continue;
            }
            if (value == "]")
            {
                bracketDepth++;
                continue;
            }
            if (value == "[")
            {
                bracketDepth--;
                continue;
            }
            if (parenDepth == 0 && bracketDepth == 0 && targets.count(value) !=
0U)
            {
                return i;
            }
```

```
        }

        return -1;
    }

    static vector<vector<Token>> splitTopLevel(const vector<Token> &slice,
const string &delimiter)
    {
        vector<vector<Token>> parts;
        vector<Token> current;
        int parenDepth = 0;
        int bracketDepth = 0;

        for (const auto &token : slice)
        {
            if (token.value == "(")
            {
                parenDepth++;
            }
            else if (token.value == ")")
            {
                parenDepth--;
            }
            else if (token.value == "[")
            {
                bracketDepth++;
            }
            else if (token.value == "]")
            {
                bracketDepth--;
            }

            if (parenDepth == 0 && bracketDepth == 0 && token.value ==
delimiter)
            {
                parts.push_back(current);
                current.clear();
                continue;
            }

            current.push_back(token);
        }

        parts.push_back(current);
        return parts;
    }

    static size_t consumeTerm(const vector<Token> &slice, size_t start)
    {
        if (start >= slice.size())
        {
            return start;
        }

        const string &value = slice[start].value;
        if (value == "(" || value == "[")
        {
            const string closeToken = value == "(" ? ")" : "]";
            int depth = 0;
            for (size_t i = start; i < slice.size(); ++i)
            {
                if (slice[i].value == value)
                {
                    depth++;
```

```
                }
                else if (slice[i].value == closeToken)
                {
                    depth--;
                    if (depth == 0)
                    {
                        return i + 1;
                    }
                }
            }
            return slice.size();
        }

        return start + 1;
    }

    static vector<vector<Token>> splitApplicationTerms(const vector<Token>
&slice)
    {
        vector<vector<Token>> terms;
        size_t index = 0;
        while (index < slice.size())
        {
            size_t next = consumeTerm(slice, index);
            if (next <= index)
            {
                break;
            }
            terms.emplace_back(slice.begin()    +    static_cast<long>(index),
slice.begin() + static_cast<long>(next));
            index = next;
        }
        return terms;
    }

    static vector<Token> flattenLines(size_t start, size_t end, const
vector<LogicalLine> &allLines)
    {
        vector<Token> merged;
        for (size_t i = start; i < end; ++i)
        {
            merged.insert(merged.end(),            allLines[i].tokens.begin(),
allLines[i].tokens.end());
        }
        return merged;
    }

    size_t findBlockEnd(size_t start, int parentIndent) const
    {
        size_t index = start;
        while (index < lines.size() && lines[index].indent > parentIndent)
        {
            ++index;
        }
        return index;
    }

    void addError(const vector<Token> &context, const string &message)
    {
        if (!context.empty())
        {
            semanticErrors.push_back("Line " + to_string(context.front().line)
+ ", column " + to_string(context.front().column) + ": " + message);
        }
```

```
        else
        {
            semanticErrors.push_back(message);
        }
    }

    void addSymbol(const string &scope, const string &name, const string &type,
int line)
    {
        symbols.push_back({scope, name, type, line});
    }

    static string join(const vector<string> &parts, const string &separator)
    {
        string result;
        for (size_t i = 0; i < parts.size(); ++i)
        {
            if (i != 0U)
            {
                result += separator;
            }
            result += parts[i];
        }
        return result;
    }

    string normalizeTypeTokens(const vector<Token> &slice) const
    {
        if (slice.empty())
        {
            return "Unknown";
        }

        if (isFullyWrapped(slice, "(", ")"))
        {
            vector<Token> inner(slice.begin() + 1, slice.end() - 1);
            if (inner.empty())
            {
                return "()";
            }

            auto parts = splitTopLevel(inner, ",");
            if (parts.size() > 1U)
            {
                vector<string> normalized;
                for (const auto &part : parts)
                {
                    normalized.push_back(normalizeTypeTokens(part));
                }
                return "(" + join(normalized, ", ") + ")";
            }
            return normalizeTypeTokens(inner);
        }

        auto terms = splitApplicationTerms(slice);
        if (terms.size() > 1U)
        {
            vector<string> normalized;
            for (const auto &term : terms)
            {
                normalized.push_back(normalizeTypeTokens(term));
            }
            return join(normalized, " ");
        }
```

```
        return slice.front().value;
    }

    pair<vector<string>,    string>    parseFunctionType(const    vector<Token>
&typeTokens) const
    {
        vector<string> parts;
        size_t start = 0;

        while (start < typeTokens.size())
        {
            vector<Token> rest(typeTokens.begin() + static_cast<long>(start),
typeTokens.end());
            int arrowIndex = findTopLevelToken(rest, {"->"});
            if (arrowIndex < 0)
            {
                parts.push_back(normalizeTypeTokens(rest));
                break;
            }

            vector<Token> chunk(rest.begin(), rest.begin() + arrowIndex);
            parts.push_back(normalizeTypeTokens(chunk));
            start += static_cast<size_t>(arrowIndex) + 1;
        }

        if (parts.empty())
        {
            return {{}, "Unknown"};
        }

        string returnType = parts.back();
        parts.pop_back();
        return {parts, returnType};
    }

    static bool isNumericType(const string &type)
    {
        return type == "Int" || type == "Double" || type == "Nat" || type ==
"Integer" || type == "Bits8" ||
               type  ==  "Numeric"  ||  type  ==  "IntLiteral"  ||  type  ==
"DoubleLiteral";
    }

    static bool isIntegralType(const string &type)
    {
        return type == "Int" || type == "Nat" || type == "Integer" || type ==
"Bits8" || type == "IntLiteral";
    }

    static bool isIOType(const string &type)
    {
        return type == "IO ()" || (type.size() > 3U && type.rfind("IO ", 0) ==
0);
    }

    static bool isListLike(const string &type)
    {
        return type.rfind("List ", 0) == 0 || type.rfind("Vect ", 0) == 0;
    }

    static string listElementType(const string &type)
    {
        if (type.rfind("List ", 0) == 0)
```

```
        {
            return type.substr(5);
        }
        if (type.rfind("Vect ", 0) == 0)
        {
            size_t firstSpace = type.find(' ');
            size_t secondSpace = type.find(' ', firstSpace + 1);
            if (secondSpace != string::npos)
            {
                return type.substr(secondSpace + 1);
            }
        }
        return "Unknown";
    }

    static vector<string> parseTupleType(const string &type)
    {
        vector<string> result;
        if (type.size() < 2U || type.front() != '(' || type.back() != ')')
        {
            return result;
        }

        string inner = type.substr(1, type.size() - 2);
        string current;
        int depth = 0;
        for (char ch : inner)
        {
            if (ch == '(')
            {
                depth++;
            }
            else if (ch == ')')
            {
                depth--;
            }

            if (ch == ',' && depth == 0)
            {
                if (!current.empty() && current.front() == ' ')
                {
                    current.erase(current.begin());
                }
                result.push_back(current);
                current.clear();
                continue;
            }

            current += ch;
        }

        if (!current.empty())
        {
            if (current.front() == ' ')
            {
                current.erase(current.begin());
            }
            result.push_back(current);
        }
        return result;
    }

    static bool areTypesCompatible(const string &expected, const string &actual)
```

```
    {
        if (expected == "Unknown" || actual == "Unknown")
        {
            return true;
        }

        if (expected == actual)
        {
            return true;
        }

        if (expected == "Double" && (actual == "IntLiteral" || actual ==
"DoubleLiteral" || actual == "Int" || actual == "Nat" || actual == "Integer"
|| actual == "Bits8"))
        {
            return true;
        }

        if (isIntegralType(expected) && actual == "IntLiteral")
        {
            return true;
        }

        if (isNumericType(expected) && isNumericType(actual))
        {
            return true;
        }

        if (isListLike(expected) && isListLike(actual))
        {
            return               areTypesCompatible(listElementType(expected),
listElementType(actual));
        }

        vector<string> expectedTuple = parseTupleType(expected);
        vector<string> actualTuple = parseTupleType(actual);
        if     (!expectedTuple.empty()     &&     expectedTuple.size()     ==
actualTuple.size())
        {
            for (size_t i = 0; i < expectedTuple.size(); ++i)
            {
                if (!areTypesCompatible(expectedTuple[i], actualTuple[i]))
                {
                    return false;
                }
            }
            return true;
        }

        return false;
    }

    static string mergeTypes(const string &left, const string &right)
    {
        if (left == right)
        {
            return left;
        }
        if (left == "Unknown")
        {
            return right;
        }
        if (right == "Unknown")
        {
```

```
            return left;
        }
        if (isNumericType(left) && isNumericType(right))
        {
            if (left == "Double" || right == "Double" || left == "DoubleLiteral"
|| right == "DoubleLiteral")
            {
                return "Double";
            }
            return "Int";
        }
        return "Unknown";
    }

    string inferAtomType(const vector<Token> &slice, const map<string, string>
&env)
    {
        if (slice.empty())
        {
            return "Unknown";
        }

        if (isFullyWrapped(slice, "(", ")"))
        {
            vector<Token> inner(slice.begin() + 1, slice.end() - 1);
            if (inner.empty())
            {
                return "()";
            }

            auto tupleParts = splitTopLevel(inner, ",");
            if (tupleParts.size() > 1U)
            {
                vector<string> tupleTypes;
                for (const auto &part : tupleParts)
                {
                    tupleTypes.push_back(inferExpressionType(part, env));
                }
                return "(" + join(tupleTypes, ", ") + ")";
            }

            return inferExpressionType(inner, env);
        }

        if (isFullyWrapped(slice, "[", "]"))
        {
            vector<Token> inner(slice.begin() + 1, slice.end() - 1);
            if (inner.empty())
            {
                return "List Unknown";
            }

            auto parts = splitTopLevel(inner, ",");
            string elementType = "Unknown";
            for (const auto &part : parts)
            {
                elementType             =             mergeTypes(elementType,
inferExpressionType(part, env));
            }
            return "List " + elementType;
        }

        if (slice.size() == 1U)
        {
```

```
            const Token &token = slice.front();
            if (token.kind == TokenKind::Number)
            {
                return token.value.find('.') != string::npos ? "DoubleLiteral"
: "IntLiteral";
            }
            if (token.kind == TokenKind::StringLiteral)
            {
                return "String";
            }
            if (token.kind == TokenKind::CharLiteral)
            {
                return "Char";
            }
            if   (token.kind   ==   TokenKind::Identifier   ||   token.kind   ==
TokenKind::TypeName)
            {
                auto localIt = env.find(token.value);
                if (localIt != env.end())
                {
                    return localIt->second;
                }
                auto globalValueIt = globalValues.find(token.value);
                if (globalValueIt != globalValues.end())
                {
                    return globalValueIt->second;
                }
                if (token.value == "Z")
                {
                    return "Nat";
                }
                if (token.value == "_")
                {
                    return "Unknown";
                }
            }
        }

        return "Unknown";
    }

    string  inferApplicationType(const  vector<vector<Token>>  &terms,  const
map<string, string> &env)
    {
        if (terms.empty())
        {
            return "Unknown";
        }

        if (terms.size() == 1U)
        {
            return inferAtomType(terms.front(), env);
        }

        if (terms.front().size() == 1U)
        {
            const string functionName = terms.front().front().value;

            if (functionName == "S")
            {
                if (terms.size() != 2U)
                {
                    addError(terms.front(),   "constructor   S   expects   one
argument");
```

```
                    return "Nat";
                }
                string argumentType = inferExpressionType(terms[1], env);
                if (!areTypesCompatible("Nat", argumentType))
                {
                    addError(terms[1], "constructor S expects argument of type
Nat, got " + argumentType);
                }
                return "Nat";
            }

            if (functionName == "cast")
            {
                if (terms.size() != 2U)
                {
                    addError(terms.front(),   "function   cast   expects   one
argument");
                }
                return "Numeric";
            }

            if (functionName == "show")
            {
                if (terms.size() != 2U)
                {
                    addError(terms.front(),   "function   show   expects   one
argument");
                }
                return "String";
            }

            auto functionIt = functionTable.find(functionName);
            if (functionIt != functionTable.end())
            {
                const FunctionType &functionType = functionIt->second;
                const size_t argumentCount = terms.size() - 1;
                if (argumentCount != functionType.parameterTypes.size())
                {
                    addError(terms.front(), "function " + functionName + "
expects " + to_string(functionType.parameterTypes.size()) +
                                                 "   argument(s),   got   "   +
to_string(argumentCount));
                }

                size_t         comparable         =         min(argumentCount,
functionType.parameterTypes.size());
                for (size_t i = 0; i < comparable; ++i)
                {
                    string actualType = inferExpressionType(terms[i + 1], env);
                    if    (!areTypesCompatible(functionType.parameterTypes[i],
actualType))
                    {
                        addError(terms[i + 1], "argument " + to_string(i + 1)
+ " of " + functionName + " has type " + actualType +
                                                      ",     expected     "     +
functionType.parameterTypes[i]);
                    }
                }
                return functionType.returnType;
            }
        }

        return "Unknown";
    }
```

```
    string inferExpressionType(const vector<Token> &slice, const map<string,
string> &env)
    {
        if (slice.empty())
        {
            return "Unknown";
        }

        if (slice.front().value == "if")
        {
            int thenIndex = findTopLevelToken(slice, {"then"});
            int elseIndex = findTopLevelToken(slice, {"else"});
            if (thenIndex < 0 || elseIndex < 0 || thenIndex >= elseIndex)
            {
                return "Unknown";
            }

            vector<Token>   condition(slice.begin()   +   1,   slice.begin()   +
thenIndex);
            vector<Token>   thenTokens(slice.begin()   +   thenIndex   +   1,
slice.begin() + elseIndex);
            vector<Token>   elseTokens(slice.begin()   +   elseIndex   +   1,
slice.end());

            string conditionType = inferExpressionType(condition, env);
            if (!areTypesCompatible("Bool", conditionType))
            {
                addError(condition, "if condition must have type Bool, got " +
conditionType);
            }

            string thenType = inferExpressionType(thenTokens, env);
            string elseType = inferExpressionType(elseTokens, env);
            if        (!areTypesCompatible(thenType,        elseType)        &&
!areTypesCompatible(elseType, thenType))
            {
                addError(slice,  "if  branches  have  incompatible  types:  "  +
thenType + " and " + elseType);
                return "Unknown";
            }
            return mergeTypes(thenType, elseType);
        }

        if (slice.front().value == "let")
        {
            int inIndex = findTopLevelToken(slice, {"in"});
            if (inIndex < 0)
            {
                return "Unknown";
            }

            vector<Token> binding(slice.begin() + 1, slice.begin() + inIndex);
            vector<Token> body(slice.begin() + inIndex + 1, slice.end());
            map<string, string> nestedEnv = env;
            analyzeLetBinding(binding, nestedEnv, "let");
            return inferExpressionType(body, nestedEnv);
        }

        if (isFullyWrapped(slice, "(", ")") || isFullyWrapped(slice, "[", "]"))
        {
            return inferAtomType(slice, env);
        }
```

```
        static const vector<set<string>> precedence = {
            {"&&", "||"},
            {">", "<", ">=", "<=", "==", "/="},
            {"++"},
            {"+", "-"},
            {"*", "/"}};

        for (const auto &level : precedence)
        {
            int opIndex = findTopLevelToken(slice, level, true);
            if (opIndex > 0 && static_cast<size_t>(opIndex + 1) < slice.size())
            {
                vector<Token> left(slice.begin(), slice.begin() + opIndex);
                vector<Token> right(slice.begin() + opIndex + 1, slice.end());
                const string op = slice[static_cast<size_t>(opIndex)].value;
                const string leftType = inferExpressionType(left, env);
                const string rightType = inferExpressionType(right, env);

                if (op == "&&" || op == "||")
                {
                    if (!areTypesCompatible("Bool", leftType))
                    {
                        addError(left, "left operand of " + op + " must be Bool,
got " + leftType);
                    }
                    if (!areTypesCompatible("Bool", rightType))
                    {
                        addError(right, "right operand of " + op + " must be
Bool, got " + rightType);
                    }
                    return "Bool";
                }

                if (op == "++")
                {
                    if     (areTypesCompatible("String",     leftType)     &&
areTypesCompatible("String", rightType))
                    {
                        return "String";
                    }
                    if (isListLike(leftType) && isListLike(rightType))
                    {
                        return leftType;
                    }
                    addError(slice, "operator ++ requires String or compatible
list operands, got " + leftType + " and " + rightType);
                    return "Unknown";
                }

                if (op == ">" || op == "<" || op == ">=" || op == "<=")
                {
                    if              (!(isNumericType(leftType)             &&
isNumericType(rightType)))
                    {
                        addError(slice,  "comparison  operator  "  +  op  +  "
requires numeric operands, got " + leftType + " and " + rightType);
                    }
                    return "Bool";
                }

                if (op == "==" || op == "/=")
                {
                    if     (!areTypesCompatible(leftType,     rightType)     &&
!areTypesCompatible(rightType, leftType))
```

```
                    {
                        addError(slice, "equality operator " + op + " requires
compatible operands, got " + leftType + " and " + rightType);
                    }
                    return "Bool";
                }

                if (!(isNumericType(leftType) && isNumericType(rightType)))
                {
                    addError(slice, "operator " + op + " requires numeric
operands, got " + leftType + " and " + rightType);
                    return "Unknown";
                }
                return mergeTypes(leftType, rightType);
            }
        }

        auto terms = splitApplicationTerms(slice);
        if (!terms.empty())
        {
            return inferApplicationType(terms, env);
        }

        return "Unknown";
    }

    void bindPattern(const vector<Token> &pattern, const string &expectedType,
map<string, string> &env, const string &scope)
    {
        if (pattern.empty())
        {
            return;
        }

        if (isFullyWrapped(pattern, "(", ")"))
        {
            vector<Token> inner(pattern.begin() + 1, pattern.end() - 1);
            if (inner.empty())
            {
                if (!areTypesCompatible("()", expectedType))
                {
                    addError(pattern, "unit pattern is incompatible with type
" + expectedType);
                }
                return;
            }

            auto parts = splitTopLevel(inner, ",");
            if (parts.size() > 1U)
            {
                vector<string> tupleTypes = parseTupleType(expectedType);
                if (!tupleTypes.empty() && tupleTypes.size() != parts.size())
                {
                    addError(pattern, "tuple pattern arity does not match type
" + expectedType);
                }

                for (size_t i = 0; i < parts.size(); ++i)
                {
                    string partType = i < tupleTypes.size() ? tupleTypes[i] :
"Unknown";
                    bindPattern(parts[i], partType, env, scope);
                }
                return;
```

```
            }

            bindPattern(inner, expectedType, env, scope);
            return;
        }

        auto terms = splitApplicationTerms(pattern);
        if (terms.size() > 1U && terms.front().size() == 1U &&
terms.front().front().value == "S")
        {
            if (!areTypesCompatible("Nat", expectedType))
            {
                addError(pattern, "pattern S ... expects Nat, got " +
expectedType);
            }
            bindPattern(terms[1], "Nat", env, scope);
            return;
        }

        if (pattern.size() == 1U)
        {
            const Token &token = pattern.front();
            if (token.value == "_")
            {
                return;
            }

            if (token.kind == TokenKind::Identifier && !token.value.empty() &&
isupper(static_cast<unsigned char>(token.value.front())))
            {
                if (token.value == "Z" && !areTypesCompatible("Nat",
expectedType))
                {
                    addError(pattern, "pattern Z expects Nat, got " +
expectedType);
                }
                return;
            }

            if (token.kind == TokenKind::Identifier || token.kind ==
TokenKind::TypeName)
            {
                env[token.value] = expectedType;
                addSymbol(scope, token.value, expectedType, token.line);
                return;
            }

            string literalType = inferExpressionType(pattern, env);
            if (!areTypesCompatible(expectedType, literalType))
            {
                addError(pattern, "pattern literal has type " + literalType +
", expected " + expectedType);
            }
        }
    }

    void analyzeLetBinding(const vector<Token> &binding, map<string, string>
&env, const string &scope)
    {
        int equalIndex = findTopLevelToken(binding, {"="});
        if (equalIndex < 0 || binding.empty())
        {
            addError(binding, "invalid let binding");
            return;
```

```
        }

        string name = binding.front().value;
        int colonIndex = findTopLevelToken(vector<Token>(binding.begin(),
binding.begin() + equalIndex), {":"});
        string declaredType = "Unknown";
        if (colonIndex >= 0)
        {
            vector<Token> typeTokens(binding.begin() + colonIndex + 1,
binding.begin() + equalIndex);
            declaredType = normalizeTypeTokens(typeTokens);
        }

        vector<Token> valueTokens(binding.begin() + equalIndex + 1,
binding.end());
        string actualType = inferExpressionType(valueTokens, env);
        string finalType = declaredType == "Unknown" ? actualType :
declaredType;

        if (declaredType != "Unknown" && !areTypesCompatible(declaredType,
actualType))
        {
            addError(valueTokens, "value of let binding " + name + " has type
" + actualType + ", expected " + declaredType);
        }

        env[name] = finalType;
        addSymbol(scope, name, finalType, binding.front().line);
    }

    string analyzeCaseBlock(size_t start, size_t end, const map<string, string>
&env, const string &scope)
    {
        const auto &caseLine = lines[start];
        int ofIndex = findTopLevelToken(caseLine.tokens, {"of"});
        if (ofIndex < 0)
        {
            addError(caseLine.tokens, "case expression must contain 'of'");
            return "Unknown";
        }

        vector<Token> subjectTokens(caseLine.tokens.begin() + 1,
caseLine.tokens.begin() + ofIndex);
        string subjectType = inferExpressionType(subjectTokens, env);

        string resultType = "Unknown";
        size_t branchIndex = start + 1;
        while (branchIndex < end)
        {
            const auto &branchLine = lines[branchIndex];
            if (branchLine.indent <= caseLine.indent)
            {
                break;
            }

            size_t branchEnd = branchIndex + 1;
            while (branchEnd < end && lines[branchEnd].indent >
branchLine.indent)
            {
                ++branchEnd;
            }

            int arrowIndex = findTopLevelToken(branchLine.tokens, {"=>"});
            if (arrowIndex < 0)
```

```
            {
                addError(branchLine.tokens, "case branch must contain '=>'");
                branchIndex = branchEnd;
                continue;
            }

            map<string, string> branchEnv = env;
            vector<Token>             patternTokens(branchLine.tokens.begin(),
branchLine.tokens.begin() + arrowIndex);
            bindPattern(patternTokens, subjectType, branchEnv, scope);

            vector<Token> bodyTokens(branchLine.tokens.begin() + arrowIndex +
1, branchLine.tokens.end());
            string branchType = bodyTokens.empty()
                                    ?
inferExpressionType(flattenLines(branchIndex + 1, branchEnd, lines), branchEnv)
                                    :          inferExpressionType(bodyTokens,
branchEnv);

            if (resultType == "Unknown")
            {
                resultType = branchType;
            }
            else    if    (!areTypesCompatible(resultType,    branchType)    &&
!areTypesCompatible(branchType, resultType))
            {
                addError(branchLine.tokens, "case branches have incompatible
types: " + resultType + " and " + branchType);
                resultType = "Unknown";
            }

            branchIndex = branchEnd;
        }

        return resultType;
    }

    string analyzeDoBlock(size_t start, size_t end, const map<string, string>
&incomingEnv, const string &scope)
    {
        map<string, string> env = incomingEnv;
        string blockType = "IO ()";

        size_t statementIndex = start;
        while (statementIndex < end)
        {
            const auto &line = lines[statementIndex];
            size_t statementEnd = statementIndex + 1;
            while   (statementEnd   <   end   &&   lines[statementEnd].indent   >
line.indent)
            {
                ++statementEnd;
            }

            if (!line.tokens.empty() && line.tokens.front().value == "let")
            {
                analyzeLetBinding(vector<Token>(line.tokens.begin()    +    1,
line.tokens.end()), env, scope);
                statementIndex = statementEnd;
                continue;
            }

            string statementType = "Unknown";
            if (!line.tokens.empty() && line.tokens.front().value == "case")
```

```
            {
                statementType = analyzeCaseBlock(statementIndex, statementEnd,
env, scope);
            }
            else
            {
                vector<Token> expressionTokens = line.tokens;
                if (statementEnd > statementIndex + 1)
                {
                    auto   nested   =   flattenLines(statementIndex   +   1,
statementEnd, lines);
                    expressionTokens.insert(expressionTokens.end(),
nested.begin(), nested.end());
                }
                statementType = inferExpressionType(expressionTokens, env);
            }

            if (!isIOType(statementType) && statementType != "Unknown")
            {
                addError(line.tokens, "statement inside do-block must have IO
type, got " + statementType);
            }

            blockType = statementType;
            statementIndex = statementEnd;
        }

        return blockType;
    }

    void registerBuiltins()
    {
        functionTable["putStrLn"] = {{"String"}, "IO ()", 0};
        functionTable["show"] = {{"Numeric"}, "String", 0};
        functionTable["cast"] = {{"Numeric"}, "Numeric", 0};
        functionTable["insert"] = {{"String", "Int", "SortedMap String Int"},
"SortedMap String Int", 0};
        functionTable["forLoop"] = {{"Nat"}, "IO ()", 0};
        globalValues["empty"] = "SortedMap String Int";
    }

    void collectGlobalSignatures()
    {
        for (const auto &line : lines)
        {
            if (line.indent != 1 || hasLexicalError(line))
            {
                continue;
            }

            int colonIndex = findTopLevelToken(line.tokens, {":"});
            int equalIndex = findTopLevelToken(line.tokens, {"="});
            if (colonIndex <= 0 || equalIndex >= 0)
            {
                continue;
            }

            string name = line.tokens.front().value;
            vector<Token>  typeTokens(line.tokens.begin()  +  colonIndex  +  1,
line.tokens.end());
            auto parsed = parseFunctionType(typeTokens);
            functionTable[name]      =      {parsed.first,      parsed.second,
line.lineNumber};
```

```
            addSymbol("global",     name,     normalizeTypeTokens(typeTokens),
line.lineNumber);
        }
    }

    void analyzeFunctionDefinitions()
    {
        size_t index = 0;
        while (index < lines.size())
        {
            const auto &line = lines[index];
            if (line.indent != 1 || hasLexicalError(line))
            {
                ++index;
                continue;
            }

            int equalIndex = findTopLevelToken(line.tokens, {"="});
            if (equalIndex <= 0)
            {
                ++index;
                continue;
            }

            string functionName = line.tokens.front().value;
            size_t blockEnd = findBlockEnd(index + 1, line.indent);
            map<string, string> env;

            auto functionIt = functionTable.find(functionName);
            FunctionType declaredType;
            if (functionIt != functionTable.end())
            {
                declaredType = functionIt->second;
            }

            vector<Token>       lhsTokens(line.tokens.begin()       +       1,
line.tokens.begin() + equalIndex);
            auto parameterPatterns = splitApplicationTerms(lhsTokens);
            if             (!declaredType.parameterTypes.empty()             &&
parameterPatterns.size() != declaredType.parameterTypes.size())
            {
                addError(line.tokens, "definition of " + functionName + " has
" + to_string(parameterPatterns.size()) +
                                                "    parameter    pattern(s),    but
signature declares " + to_string(declaredType.parameterTypes.size()));
            }

            for (size_t i = 0; i < parameterPatterns.size(); ++i)
            {
                string expectedType = i < declaredType.parameterTypes.size() ?
declaredType.parameterTypes[i] : "Unknown";
                bindPattern(parameterPatterns[i],      expectedType,      env,
functionName);
            }

            vector<Token>  rhsInline(line.tokens.begin()  +  equalIndex  +  1,
line.tokens.end());
            string actualType = "Unknown";

            if (!rhsInline.empty())
            {
                if (rhsInline.front().value == "do")
                {
```

```
                    actualType = analyzeDoBlock(index + 1, blockEnd, env,
functionName);
                }
                else if (rhsInline.front().value == "case")
                {
                    actualType = analyzeCaseBlock(index, blockEnd, env,
functionName);
                }
                else
                {
                    vector<Token> fullExpression = rhsInline;
                    if (blockEnd > index + 1)
                    {
                        auto nested = flattenLines(index + 1, blockEnd, lines);
                        fullExpression.insert(fullExpression.end(),
nested.begin(), nested.end());
                    }
                    actualType = inferExpressionType(fullExpression, env);
                }
            }
            else if (blockEnd > index + 1)
            {
                if (!lines[index + 1].tokens.empty() && lines[index +
1].tokens.front().value == "case")
                {
                    actualType = analyzeCaseBlock(index + 1, blockEnd, env,
functionName);
                }
                else if (!lines[index + 1].tokens.empty() && lines[index +
1].tokens.front().value == "do")
                {
                    actualType = analyzeDoBlock(index + 2, blockEnd, env,
functionName);
                }
                else
                {
                    actualType = inferExpressionType(flattenLines(index + 1,
blockEnd, lines), env);
                }
            }

            if (!declaredType.returnType.empty() && declaredType.returnType !=
"Unknown" &&
                !areTypesCompatible(declaredType.returnType, actualType))
            {
                addError(line.tokens, "function " + functionName + " returns "
+ actualType + ", but signature declares " + declaredType.returnType);
            }

            index = blockEnd;
        }
    }

public:
    SemanticAnalyzer(const vector<Token> &allTokens, const vector<LogicalLine>
&logicalLines)
        : tokens(allTokens), lines(logicalLines) {}

    void analyze()
    {
        registerBuiltins();
        collectGlobalSignatures();
        analyzeFunctionDefinitions();
    }
```

```
    const vector<string> &getErrors() const
    {
        return semanticErrors;
    }

    const vector<SymbolEntry> &getSymbols() const
    {
        return symbols;
    }
};

string readInputFile()
{
    const vector<string> candidates = {
        "../../lab3/build/input.txt",
        "../lab3/build/input.txt",
        "lab3/build/input.txt",
        "input.txt",
        "build/input.txt",
        "../../lab4/build/input.txt"};

    for (const auto &path : candidates)
    {
        ifstream input(path);
        if (!input.is_open())
        {
            continue;
        }

        stringstream buffer;
        buffer << input.rdbuf();
        return buffer.str();
    }

    return {};
}

string tokenKindToString(TokenKind kind)
{
    switch (kind)
    {
    case TokenKind::Reserved:
        return "Reserved";
    case TokenKind::Identifier:
        return "Identifier";
    case TokenKind::TypeName:
        return "Type";
    case TokenKind::Number:
        return "Number";
    case TokenKind::StringLiteral:
        return "String";
    case TokenKind::CharLiteral:
        return "Char";
    case TokenKind::Operator:
        return "Operator";
    case TokenKind::Separator:
        return "Separator";
    case TokenKind::Newline:
        return "Newline";
    case TokenKind::EndOfFile:
        return "EOF";
    case TokenKind::Error:
        return "Error";
```

```
    }
    return "Unknown";
}

void printTree(const shared_ptr<AstNode> &node, const string &prefix = "", bool
isLast = true, bool isRoot = true)
{
    if (!node)
    {
        return;
    }

    if (isRoot)
    {
        cout << node->label << '\n';
    }
    else
    {
        cout << prefix;
        cout << (isLast ? "└── " : "├── ");
        cout << node->label << '\n';
    }

    for (size_t i = 0; i < node->children.size(); ++i)
    {
        bool childIsLast = i + 1 == node->children.size();
        string childPrefix = isRoot ? "" : prefix + (isLast ? "    " : "│   ");
        printTree(node->children[i], childPrefix, childIsLast, false);
    }
}

void printTokens(const vector<Token> &tokens)
{
    cout << string(70, '=') << '\n';
    cout << "TOKEN TABLE\n";
    cout << string(70, '=') << '\n';
    cout << left << setw(14) << "TYPE" << setw(28) << "VALUE" << setw(8) <<
"LINE" << setw(8) << "COLUMN" << '\n';
    cout << string(70, '-') << '\n';

    for (const auto &token : tokens)
    {
        cout << left << setw(14) << tokenKindToString(token.kind) << setw(28)
             << (token.kind == TokenKind::StringLiteral ? "\"" + token.value +
"\"" : token.kind == TokenKind::CharLiteral ? "'" + token.value + "'" :
token.value)
             << setw(8) << token.line << setw(8) << token.column << '\n';
    }
}

void printErrors(const vector<Token> &tokens, const vector<string>
&syntaxErrors)
{
    vector<string> lexicalErrors;
    for (const auto &token : tokens)
    {
        if (token.kind == TokenKind::Error)
        {
            lexicalErrors.push_back("Line " + to_string(token.line) + ", column
" + to_string(token.column) + ": " + token.value);
        }
    }

    cout << "\n" << string(70, '=') << '\n';
```

```
    cout << "LEXICAL ERRORS\n";
    cout << string(70, '=') << '\n';
    if (lexicalErrors.empty())
    {
        cout << "No lexical errors found.\n";
    }
    else
    {
        for (const auto &error : lexicalErrors)
        {
            cout << error << '\n';
        }
    }

    cout << "\n" << string(70, '=') << '\n';
    cout << "SYNTAX ERRORS\n";
    cout << string(70, '=') << '\n';
    if (syntaxErrors.empty())
    {
        cout << "No syntax errors found.\n";
    }
    else
    {
        for (const auto &error : syntaxErrors)
        {
            cout << error << '\n';
        }
    }
}

void printSemanticSymbols(const vector<SymbolEntry> &symbols)
{
    const int totalWidth = 96;
    const int scopeWidth = 18;
    const int nameWidth = 18;
    const int typeWidth = 48;
    const int lineWidth = 8;

    cout << "\n" << string(totalWidth, '=') << '\n';
    cout << "SEMANTIC SYMBOL TABLE\n";
    cout << string(totalWidth, '=') << '\n';
    cout << left << setw(scopeWidth) << "SCOPE"
         << setw(nameWidth) << "NAME"
         << setw(typeWidth) << "TYPE"
         << setw(lineWidth) << "LINE" << '\n';
    cout << string(totalWidth, '-') << '\n';

    for (const auto &entry : symbols)
    {
        cout << left << setw(scopeWidth) << entry.scope
             << setw(nameWidth) << entry.name
             << setw(typeWidth) << entry.type
             << setw(lineWidth) << entry.line << '\n';
    }
}

void printSemanticErrors(const vector<string> &semanticErrors)
{
    cout << "\n" << string(70, '=') << '\n';
    cout << "SEMANTIC ERRORS\n";
    cout << string(70, '=') << '\n';
    if (semanticErrors.empty())
    {
        cout << "No semantic errors found.\n";
```

```
        return;
    }

    for (const auto &error : semanticErrors)
    {
        cout << error << '\n';
    }
}
```